

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ
РОССИЙСКОЙ ФЕДЕРАЦИИ

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ

**«БЕЛГОРОДСКИЙ ГОСУДАРСТВЕННЫЙ
ТЕХНОЛОГИЧЕСКИЙ УНИВЕРСИТЕТ им. В. Г. ШУХОВА»
(БГТУ им. В.Г. Шухова)**

Кафедра программного обеспечения вычислительной техники и автоматизированных
систем



Лабораторная работа №5
по дисциплине: Теория информации
тема: Исследовать особенности метода арифметического кодирования

Выполнил: ст. группы ПВ-221
Дорошенко Владимир Юрьевич
Проверили: Твердохлеб В. В.

Белгород 2024

Цель работы: исследовать особенности метода арифметического кодирования

Задача №1

Построить обработчик, реализующий арифметическое кодирование. Предусмотреть возможность обработки символьных и двоичных данных. В качестве исходных данных, подлежащих обработке, использовать последовательности из работы №2.

Код программы:

```
from sys import getsizeof
from fractions import Fraction
import copy

def message_to_code(message, x):
    alphabet = {}
    decode_table = {}

    message += "\0"
    max_denominator = 10 ** int(len(message) / x)

    for char in message:
        if char in alphabet.keys():
            alphabet[char] += 1
        else:
            alphabet[char] = 1
    buf = (Fraction(0), Fraction(0))
    for char in alphabet.keys():
        alphabet[char] /= Fraction(len(message))
        decode_table[char] = (buf[1], buf[1] + alphabet[char])
        buf = decode_table[char]

    buf = (Fraction(0), Fraction(0))
    for ch in message:
        t = decode_table[ch]
        if buf == (0, 0):
            buf = t
        else:
            left = buf[0].limit_denominator(max_denominator)
            right = buf[1].limit_denominator(max_denominator)
            buf = (left + (right - left) * t[0], left + (right - left) * t[1])

    return ((buf[0] + buf[1]) / 2).limit_denominator(max_denominator), decode_table

def code_to_message(code, decode_table):
    message = ''

    while code:
        t = ('', (Fraction(0), Fraction(0)))
        for char in decode_table.keys():
            if decode_table[char][0] <= code and code <= decode_table[char][1]:
                t = (char, decode_table[char])
                code -= decode_table[char][0]

        if t[0] == "\0":
            return message

    message += t[0]
    code = (code - t[1][0]) / (t[1][1] - t[1][0])
    return message
```

```
if name == 'main':
list_of_messages = ["в чашах юга жил бы цитрус? Да, но фальшивый экземпляр!", "Victoria nulla est, Quam quae confessos animo quoque subjugat hostes"]
for string in list_of_messages:

code, table = message_to_code(string, 1) print(code)
for index, char in enumerate(table):
if index % 3 == 0:
print()
print(f"'{char}': ({table[char][0]}; {table[char][1]})".ljust(20, " "), end=" | ") print()
print(code_to_message(code, table))
print(len(string) / (sizeof(code.numerator) + sizeof(code.denominator) - 48), "\n")
```

24263921638147316284877138470953910477554138439761178/9556163637945604181497520647487770577619866986794790493

'в': (0; 2/55)	' ': (2/55; 1/5)	'ч': (1/5; 12/55)	
'а': (12/55; 17/55)	'щ': (17/55; 18/55)	'х': (18/55; 19/55)	
'ю': (19/55; 4/11)	'г': (4/11; 21/55)	'ж': (21/55; 2/5)	
'и': (2/5; 5/11)	'л': (5/11; 28/55)	'б': (28/55; 29/55)	
'ы': (29/55; 31/55)	'ц': (31/55; 32/55)	'т': (32/55; 3/5)	
'р': (3/5; 7/11)	'у': (7/11; 36/55)	'с': (36/55; 37/55)	
'?': (37/55; 38/55)	'д': (38/55; 39/55)	', ': (39/55; 8/11)	
'н': (8/11; 41/55)	'о': (41/55; 42/55)	'ф': (42/55; 43/55)	
'ь': (43/55; 4/5)	'ш': (4/5; 9/11)	'й': (9/11; 46/55)	
'э': (46/55; 47/55)	'к': (47/55; 48/55)	'з': (48/55; 49/55)	
'е': (49/55; 10/11)	'м': (10/11; 51/55)	'п': (51/55; 52/55)	
'я': (52/55; 53/55)	'!'': (53/55; 54/55)	'0': (54/55; 1)	

в чащах юга жил бы цитрус? Да, но фальшивый экземпляр!

Результат работы программы:

156607142119066966474881474699390556400730735587563826981699050376/630637124373498538105354374293545660942271603014446281901949715146047

'V': (0; 1/69)	'i': (1/69; 4/69)	'c': (4/69; 2/23)	
't': (2/23; 10/69)	'o': (10/69; 16/69)	'r': (16/69; 17/69)	
'a': (17/69; 1/3)	' ': (1/3; 32/69)	'n': (32/69; 35/69)	
'u': (35/69; 14/23)	'l': (14/23; 44/69)	'e': (44/69; 49/69)	
's': (49/69; 56/69)	', ': (56/69; 19/23)	'Q': (19/23; 58/69)	
'm': (58/69; 20/23)	'q': (20/23; 21/23)	'f': (21/23; 64/69)	
'b': (64/69; 65/69)	'j': (65/69; 22/23)	'g': (22/23; 67/69)	
'h': (67/69; 68/69)	'0': (68/69; 1)		

Victoria nulla est, Quam quae confessos animo quoque subjugat hostes

Оценка полученных кодов для 1
сообщения:

$$B=n\cdot\eta=166$$
$$B'=88$$
$$K = B / B' = 1.8864$$

Оценка полученных кодов для 2 сообщения:

$$B = n \cdot \eta = 109$$

$$B' = 92$$

$$K = B / B' = 1.848$$

Вывод: в ходе лабораторной работы изучили теоретический материал про канальные матрицы, научились решать задания по заданной теме

Вывод

В ходе выполнения лабораторной работы был написан обработчик, реализующий метод арифметического кодирования. В качестве тестовых данных были использованы строки из второй лабораторной работы. На их примере мы увидели, что метод арифметического кодирования обладает большим коэффициентом сжатия в сравнении с методами Хаффмена и Шеннона – Фано. Так же данный алгоритм может сжимать сообщения, записанные двоичным сообщением